

컴퓨터비전_인공지능(AI)개론

14. 사전 학습 모델_VGG/ResNet

- 다양한 파인튜닝 전략의 장단점을 이해하고 실험으로 검증할 수 있다
- 상황에 맞는 최적의 전이학습 전략을 선택할 수 있다
- 실험설계와 결과 해석 능력을 배양한다

▶ 3가지 핵심 전략

1. 완전 동결 (Full Freeze)

ImageNet Backbone → New Classifier

백본 전체를 고정하고 분류기만 학습합니다.

장점: 빠른 학습, 과적합 방지 효과

단점: 도메인 차이가 클 때 성능 제한

2. 부분 해제 (Partial Fine-tuning)

Early Layers → Last k Blocks → Classifier

초반 층을 고정하고 후반 k개 블록만 학습합니다.

장점: 균형잡힌 접근, 안정적

단점: k 값 선택이 매우 중요

3. 전체 해제 (Full Fine-tuning)

All Layers → Classifier

모든 층을 학습 가능 상태로 전환합니다.

장점: 최대 적응력과 유연성

단점: 과적합 위험, 느린 학습

실험 조건

- 모델: ResNet-50 (ImageNet pre-trained)
- 데이터셋: Custom dataset (5 classes, 500 images/class)
- Train/Val split: 80/20
- Epochs: 30
- Optimizer: SGD with momentum 0.9
- Base Learning Rate: 0.01

측정 지표

- Validation Accuracy
- Training time per epoch
- Overfitting gap (Train acc - Val acc)

▶ 비교 대상 전략

01

완전 동결

백본 전체 고정 + 분류기만 학습

02

부분 해제

처음 3개 stage 고정 + 마지막 stage와 분류기
학습

03

전체 해제

모든 층 파인튜닝

▶ 3가지 전략의 코드 구현

```
import torch
import torch.nn as nn
from torchvision import models

# Strategy 1: Full Freeze
def setup_full_freeze(model):
    for param in model.parameters():
        param.requires_grad = False
    # Only classifier trainable
    model.fc = nn.Linear(model.fc.in_features, num_classes)
    return model
```

▶ 3가지 전략의 코드 구현

```
# Strategy 2: Partial Fine-tuning (last k blocks)
def setup_partial_finetune(model, k=1):
    # Freeze all first
    for param in model.parameters():
        param.requires_grad = False

    # Unfreeze last k blocks (ResNet stages)
    if k >= 1:
        for param in model.layer4.parameters():
            param.requires_grad = True
    if k >= 2:
        for param in model.layer3.parameters():
            param.requires_grad = True

    # New classifier
    model.fc = nn.Linear(model.fc.in_features, num_classes)
    return model
```

▶ 3 가지 전략의 코드 구현

```
# Strategy 3: Full Fine-tuning
def setup_full_finetune(model):
    # All layers trainable
    for param in model.parameters():
        param.requires_grad = True
    model.fc = nn.Linear(model.fc.in_features, num_classes)
    return model
```

각 전략은 requires_grad 속성을 조작하여 어떤 파라미터를 학습할지 제어합니다.

이 간단한 메커니즘이 모델의 학습 방식을 근본적으로 변화시킵니다.

Strategy	Val Accuracy	Train Time	Overfit Gap
Full Freeze	82.5%	1.2 min	3.2%
Last 1 Block	87.3%	2.1 min	5.8%
Last 2 Blocks	89.1%	3.5 min	8.1%
Full Fine-tune	88.6%	5.2 min	12.4%

핵심 인사이트 1

부분 해제(Last 2 Blocks)가 최고 성능을 달성하며 최적의 균형점을 제공합니다.

핵심 인사이트 2

전체 파인튜닝은 과적합 갭이 크게 증가하여 일반화 성능이 저하됩니다.

핵심 인사이트 3

도메인 유사도와 데이터셋 크기에 따라 전략을 유연하게 선택해야 합니다.

▶ 데이터 셋 크기와 도메인에 따른 전략 선택

Target Data Size?

- 1) 매우 작은 (<1000) $\gg$ Full Freeze
- 2) 중간 ($1-10K$) $\gg$ Partial Fine Tuning (도메인 유사도 비교) 유사하다면, last 1-2 blocks
- 3) 큰 ($>10K$) $\gg$ Full Fine-tune (강한 regularization 필요)

실무 팁 1: 점진적 접근

작게 시작하고 필요에 따라 점진적으로 더 많은 층을 해제하는 것이 안전합니다.

실무 팁 2: 모니터링

Validation curve를 지속적으로 관찰하여 과적합 징후를 조기에 발견하세요.

실무 팁 3: Early Stopping

Early stopping 메커니즘을 활용하여 불필요한 학습을 방지하고 과적합을 예방합니다.

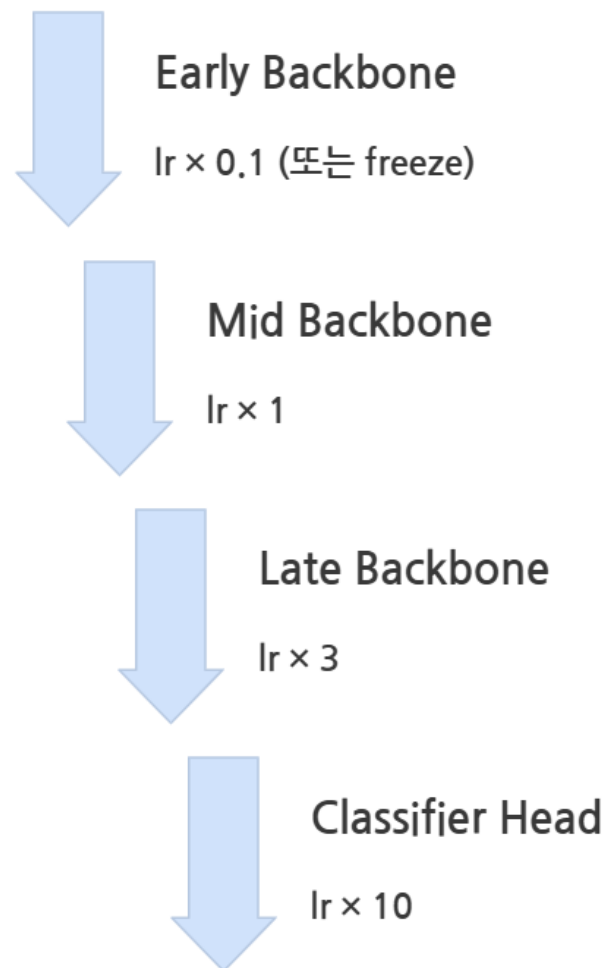
문제 상황

사전학습된 층은 이미 ImageNet에서 학습한 좋은 특징을 보유하고 있어 큰 변화가 불필요합니다. 반면 새로운 분류기는 랜덤으로 초기화되어 큰 변화가 필요합니다.

기존 단일 LR의 문제점

- LR을 크게 설정: 백본이 파괴됨 (catastrophic forgetting)
- LR을 작게 설정: 분류기 학습이 매우 느림

해결책: 차등 학습률



▶ PyTorch Parameter Groups 활용

```
# Method 1: Parameter Groups
model = models.resnet50(pretrained=True)
model.fc = nn.Linear(model.fc.in_features, num_classes)

# Define parameter groups with different LRs
params = [
    {'params': model.layer1.parameters(), 'lr': 1e-4}, # Early layers
    {'params': model.layer2.parameters(), 'lr': 3e-4},
    {'params': model.layer3.parameters(), 'lr': 1e-3},
    {'params': model.layer4.parameters(), 'lr': 3e-3}, # Late layers
    {'params': model.fc.parameters(), 'lr': 1e-2}      # Head (10x)
]

optimizer = torch.optim.SGD(params, momentum=0.9)
```

```
# Method 2: Using learning rate schedulers
def get_lr_lambda(epoch):
    # Different decay for different layers
    if epoch < 10:
        return 1.0
    elif epoch < 20:
        return 0.1
    else:
        return 0.01

# Apply different schedulers to different param groups
```

- ❏ 핵심 포인트: `params` 리스트에서 각 층별로 다른 학습률을 지정할 수 있습니다. 이를 통해 사전학습된 지식을 보존하면서도 새로운 태스크에 효과적으로 적응할 수 있습니다.

실험 설정

- Base LR: 0.001
- 비교 대상:
 1. Uniform: 모든 층에 동일하게 0.001
 2. Differential: Head 0.01, Late 0.003, Mid 0.001, Early 0.0001

결과 예시

Approach	Epoch 10	Epoch 20	Final	Convergence
Uniform	75.2%	83.1%	86.3%	Epoch 25
Differential	81.5%	88.7%	91.2%	Epoch 18

빠른 수렴

차등 학습률은 7 epoch 더 빠르게 수렴됨

높은 최종 성능

최종 정확도가 4.9% 향상됨

백분 보존

사전학습된 특징을 효과적으로 보존함

```

class ProgressiveUnfreeze:
    def __init__(self, model, stages, epochs_per_stage):
        self.model = model
        self.stages = stages # [layer1, layer2, layer3, layer4, fc]
        self.epochs_per_stage = epochs_per_stage
        self.current_stage = -1 # Start with all frozen

    def step(self, epoch):
        stage_idx = epoch // self.epochs_per_stage
        if stage_idx > self.current_stage and stage_idx < len(self.stages):
            # Unfreeze next stage
            for param in self.stages[stage_idx].parameters():
                param.requires_grad = True
            self.current_stage = stage_idx
            print(f"Unfroze stage {stage_idx}")

# Usage
unfreeze_scheduler = ProgressiveUnfreeze(
    model,
    [model.layer4, model.layer3, model.layer2, model.layer1],
    epochs_per_stage=5
)

```

Epoch 1-5
(Head만 학습)

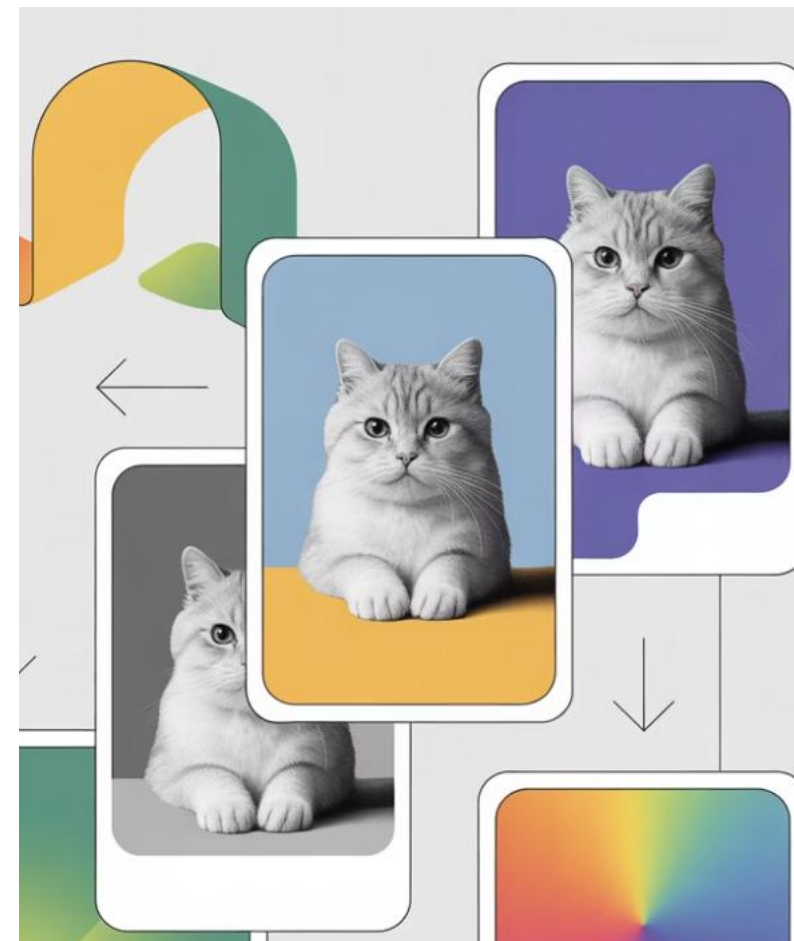
Epoch 6-10
(Head + Layer4)

Epoch 11-15
(Head + Layer4+ Layer3)

Epoch 16+
(점진적으로 모든 층 해제)

▶ 데이터 증강의 목적

- 과적합 방지: 모델이 훈련 데이터를 암기하는 것을 막습니다
- 데이터 다양성 증가: 제한된 데이터로도 다양한 변형을 학습합니다
- 불변성 학습: 회전, 크기 변화 등에 강인한 모델을 만듭니다



Weak Augmentation

- RandomHorizontalFlip
- Random crop (minimal)
- Normalize

Medium Augmentation

- RandomRotation ($\pm 15^\circ$)
- ColorJitter
- RandomResizedCrop

Strong Augmentation

- RandAugment / AutoAugment
- Cutout / Random Erasing
- MixUp / CutMix

▶ 3가지 증강 강도의 Pytorch 구현

```
from torchvision import transforms

# Weak Augmentation
weak_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

▶ 3가지 증강 강도의 Pytorch 구현

```
# Medium Augmentation
medium_transform = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2,
                           saturation=0.2, hue=0.1),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

▶ 3가지 증강 강도의 Pytorch 구현

```
# Strong Augmentation
from torchvision.transforms import autoaugment

strong_transform = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.6, 1.0)),
    autoaugment.AutoAugment(),
    transforms.RandomErasing(p=0.5),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

각 증강 레벨은 데이터의 변형 강도를 조절합니다. **Weak**는 보수적, **Medium**은 균형적, **Strong**은 공격적인 변형을 적용합니다.

▶ 실험 매트릭스 분석

Dataset Size	Weak Aug	Medium Aug	Strong Aug
500 samples	78.3% / 9.2%	82.1% / 5.6%	79.8% / 3.1%
2000 samples	85.7% / 6.8%	88.9% / 4.2%	89.5% / 2.8%
10000 samples	91.2% / 4.1%	92.8% / 2.9%	94.3% / 1.5%

⋮ 형식: Val Acc / Overfit Gap



작은 데이터셋

Medium이 최적입니다. Strong 증강은 오히려 underfitting을 유발합니다.



중간 데이터셋

Strong 증강의 효과가 나타나기 시작하며 과적합이 크게 감소합니다.



큰 데이터셋

Strong 증강으로 최고 성능을 달성하며 일반화 능력이 극대화됩니다.

▶ 주의사항

주의사항 1: 분포 왜곡

과도한 증강은 원본 데이터의 분포를 크게 변형시켜 오히려 성능을 저하시킬 수 있습니다. 특히 의료 영상처럼 미세한 디테일이 중요한 도메인에서는 신중해야 합니다.

주의사항 2: 도메인 고려

자연 이미지와 달리 의료 영상, 위성 영상 등 특수 도메인은 증강 기법 선택에 더욱 신중을 기해야 합니다. 도메인 전문가의 검토가 권장됩니다.

주의사항 3: Validation 세트

Validation 세트에는 증강을 적용하지 않아야 실제 성능을 정확히 평가할 수 있습니다. 테스트 시에도 마찬가지로입니다.

▶ Small Dataset 정의

- 클래스당 < 100 samples
- 전체 < 1000 samples
- Pre-training 데이터와 도메인 차이가 큼
- 주요 문제점
 - 과적합 위험
 - 극대화일반화 능력 저하
 - 클래스 불균형 가능성
 - 검증 전략의 어려움

해결 전략 프레임워크



점진적 학습

단계별로 층을 해제하며 안정적으로 학습



강한 정규화

Dropout, Weight Decay 등 적극 활용



신중한 증강

Medium 수준의 균형잡힌 증강



Cross-validation

K-fold로 안정적인 성능 평가

▶ 2단계 점진적 학습 전략

Phase 1: Head Pre-training (Epochs 1-10)

전략: 백본 완전 동결

- 분류기만 높은 LR($1e-3$)로 학습
- Medium 증강 적용
- Dropout 0.5로 강한 정규화
- 빠르게 분류기를 task에 적응

Phase 2: Gradual Unfreezing (Epochs 11-30)

전략: 마지막 stage 해제

- 차등 학습률: Layer4 ($1e-4$), FC ($1e-3$)
- 증강 강도 약간 감소
- Early stopping (patience=5)
- 세밀한 task 적응

▶ 2단계 점진적 학습 전략

```
# Phase 1: Freeze backbone completely
for param in model.parameters():
    param.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, num_classes)

# Step 2: Train with high LR
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)

# Phase 2: Unfreeze last stage
for param in model.layer4.parameters():
    param.requires_grad = True

# Differential LR
params = [
    {'params': model.layer4.parameters(), 'lr': 1e-4},
    {'params': model.fc.parameters(), 'lr': 1e-3}
]
optimizer = torch.optim.Adam(params)
```

▶ 작은 데이터 셋을 위한 강력한 정규화

01

Dropout

분류기에 0.5 수준의 강한 드롭아웃

02

Weight Decay

L2 정규화로 가중치 크기 제한

03

Label Smoothing

과신을 방지하는 부드러운 레이블

```
# 1. Dropout in Classifier
class CustomClassifier(nn.Module):
    def __init__(self, in_features, num_classes):
        super().__init__()
        self.dropout = nn.Dropout(0.5)
        self.fc1 = nn.Linear(in_features, 512)
        self.dropout2 = nn.Dropout(0.3)
        self.fc2 = nn.Linear(512, num_classes)

    def forward(self, x):
        x = self.dropout(x)
        x = F.relu(self.fc1(x))
        x = self.dropout2(x)
        return self.fc2(x)

model.fc = CustomClassifier(2048, num_classes)
```

▶ 작은 데이터 셋을 위한 강력한 정규화

01

Dropout

분류기에 0.5 수준의 강한 드롭아웃

02

Weight Decay

L2 정규화로 가중치 크기 제한

03

Label Smoothing

과신을 방지하는 부드러운 레이블

```
# 2. Weight Decay
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=1e-3,
    weight_decay=1e-4  # L2 regularization
)

# 3. Label Smoothing
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
```

▶ 상황별 참조 가이드

상황	Freeze Strategy	LR Strategy	Aug Strategy	추가 팁
Large dataset (>10K) Similar domain	Partial FT (Last 2 blocks)	Differential Head ×10	Strong	Standard training
Large dataset Different domain	Full FT	Differential Gradual unfreeze	Strong	Longer training
Medium dataset (1K-10K) Similar domain	Partial FT (Last 1-2 blocks)	Differential Head ×5	Medium-Strong	Careful monitoring
Small dataset (<1K) Similar domain	Head only → Last block	2-Phase Head first	Medium	Heavy regularization
Small dataset Different domain	Progressive Unfreeze	3-Phase Gradual	Medium	Cross-validation